

Informe de ingeniería inversa:

Taller 2

Organización:

Pontificia Universidad Javeriana

Bogotá D.C

Fecha:

9 de septiembre de 2026



Autores:

Juan David Hurtado

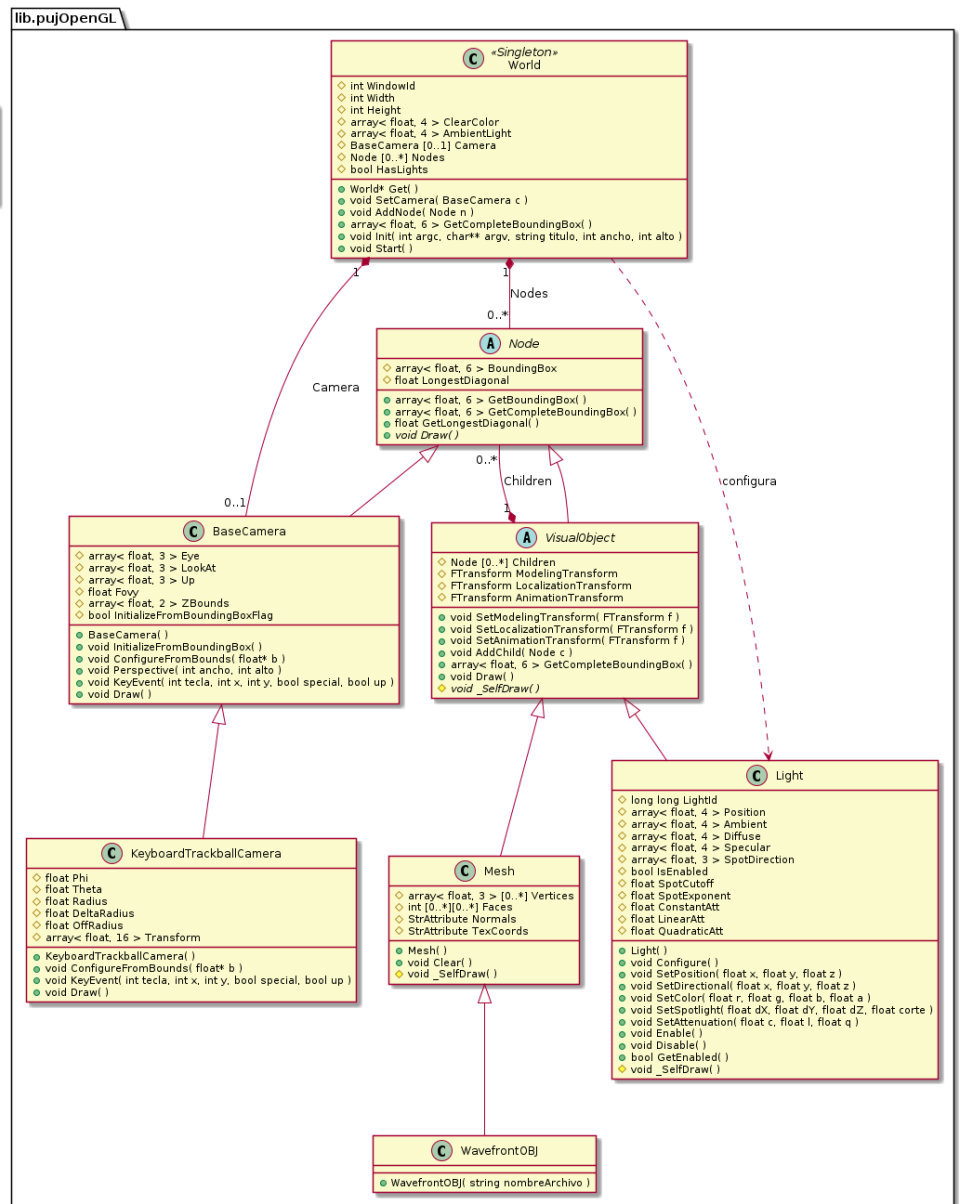
Laura Sofia Salamanca

Simón Díaz

Estructura del sistema

Diagrama de clases - lib.pujOpenGL (vista lógica)

Node: clase base abstracta del grafo de escena (caja de contenimiento y dibujo).
VisualObject: nodo con transformaciones e hijos (grafo de escena recursivo).
WavefrontOBJ: malla cargada desde archivo .obj.
World: motor GLUT, crea la ventana, registra los callbacks y ejecuta el bucle principal.



Análisis de ingeniería inversa

Inicialización del programa:

- **OBJViewer - punto de entrada:**
 - Maneja lo que recibe el programa (el .obj) y qué pasa si falta el argumento.
 - Se carga el mundo (*World*), se carga la malla (objeto con *WavefrontOBJ*), se agrega la malla al mundo, se crea y configura la cámara.
 - Acá es donde se relaciona *World* - *Camera* - *Mesh*

Objeto o malla:

- **Node - Super clase de la que heredan los objetos que se pueden 'dibujar' (Malla, Camara, y Luces)**
 - Es una clase abstracta de todo lo que puede 'dibujarse' en la escena: cámaras, mallas y luces heredan de ellas.
 - Acá se declara *Draw()* como método virtual, lo que obliga a cada objeto a implementar su propia forma de dibujarse.
 - Guarda el *bounding box* (utilizado solo en malla) para calcular el max-min de cada eje (X,Y,Z).
- **VisualObject - el que relaciona objetos/nodos**
 - Hereda de *Node* y lo que hace es relacionar los nodos padres con los hijos, y relacionar nodos con nodos.
 - Tiene el método clave *Draw()* que es el que aplica las transformaciones de modelado y delega el dibujo real del objeto a *_SelfDraw* (acá llama al método *Draw* de los hijos del nodo).
- **Mesh - clase que guarda lo relacionado con la malla**
 - Guarda los vertices de la malla: puntos en X, Y, Z.
 - Guarda las caras de la malla: relaciones entre vértices para dibujar polígonos.
 - Guarda las normales: guarda los vectores perpendiculares a las caras en un punto dado. Es para indicar esa cara a donde 'apunta'. En este taller no se usa pero sirve para el manejo de luces.
 - Guarda las texturas. (no se usa por ahora para dibujar)
- **WavefontOBJ - donde se carga el .obj**
 - A partir del archivo .obj carga la malla
 - Cuando inicia con 'v': vertices
 - Cuando inicia con 'vn': normales
 - Cuando inicia con 'vt': texturas

Cámara:

- **BaseCamara - super clase de las cámaras:**
 - Guarda *Eye* (donde está la cámara), *LookAt* (hacia dónde mira) y *Up* (qué dirección es arriba para la cámara).
 - Utiliza el BoundingBox (la caja donde se encuentra el objeto) de la malla cargada. A partir de esos números ubica la cámara:
 - *LookAt*: centro del bounding box en X,Y y Z
 - *Eye*: centro del bounding box + 3 veces la distancia al borde (esto para que el objeto 'quepa' en la cámara y se pueda observar bien).
 - *Up*: se va recalculando para que quede perpendicular a la dirección de la vista.
 - *Draw* utiliza *gluLookAt(Eye, LookAt, Up)* para configurar la matriz de vista de OpenGL para el frame
- **KeyboardTrackballCamera - cómo se controla la cámara con el teclado:**
 - Usa *Phi*, *Theta* y *Radius* para ubicar la cámara alrededor del punto que mira. *Phi* maneja la rotación vertical (en Y), *Theta* la horizontal (en X), y *Radius* mira que tan lejano está el objeto.
 - *KeyEvent* manejan los eventos con el teclado y a partir de ahí se decide sobre que se trabaja (*Phi* arriba/abajo, *Theta* izquierda/derecha, *Radius* (+/-)).

- Update: por cada evento se actualiza la matriz de transformación para ver si es una rotación o una traslación.
- Reset: calcula el radio y reinicia la matriz de transformación.
- Draw: acá se dibuja en base a la matriz transformada.

World:

- **World - clase que orquesta todo**
 - **Singleton:** solo existe una instancia de World en todo el programa.
 - Como GLUT en los callbacks no acepta métodos de clase, World tiene funciones estáticas (**_CB_***) que solo hacen Get() y reenvían la llamada al método de instancia real (**_cb_***). Esta facilidad también es consecuencia del Singleton porque la llamada no se confunde entre objetos (solo existe uno).
 - Los métodos de instancia real (**_cb_***) son los que se encargan de llamar al KeyEvent. Start() registra todos los callbacks y arranca el glutMainLoop() que es el que permite 'capturar' los eventos del teclado.

Modificaciones realizadas

1. Creación clase MouseTrackballCamera: en esta clase se manejan todos los eventos del *mouse* que se generan. Es la encargada de actualizar la matriz de transformaciones (traslaciones y rotaciones) a partir de los eventos.
2. En el **OBJViewer** se reemplaza la cámara KeyTrackballCamera por la MouseTrackballCamera.
3. Se generaron los siguientes métodos y funciones en World que manejan los eventos específicos de la siguiente forma:
 - a. **_cb_entry - _CB Entry:** Este callback se llama el momento de ingresar o salir de la ventana. La idea inicial que tuvimos fue inicializar ahí las variables de posición *x* y *y* al momento de entrar a la ventana, pero nos dimos cuenta que este callback solo recibe un parámetro *state* que nos indica si está saliendo o entrando de la ventana. Es por esto que decidimos hacer lo contrario, setear el valor por defecto a estas variables en el momento en que el usuario sale de la ventana.
 - b. **_cb_passive_mouse_motion - _CB PassiveMouseMotion:** El movimiento pasivo del *mouse* cambia de color el objeto cuando el *mouse* esta encima del objeto (lo cambia a color azul).
 - Recibe la posición del *mouse* en X, Y
 - Verifica por medio de *glReadPixels* se identifica si el píxel donde esta el *mouse* es fondo (negro) o es de otro color (diferente a negro). Si no es el fondo, entonces pinta el objeto de color azul.
 - c. **_cb_mouse_pressed - _CB_MOUSE_PRESSED - click derecho:** Cuando se oprime el botón derecho y se mueve el *mouse*, se hace una traslación en X y Y del objeto.
 - Recibe la posición del *mouse* X, Y
 - Verifica que el botón derecho esté oprimido

- Calcula la diferencia entre píxeles (tanto en X como en Y) para saber a donde trasladar el objeto (si a la derecha o a la izquierda).
 - Se crea la operación matricial correspondiente a la traslación (en X y en Y) en la matriz de transformación.
 - Se calcula una sensibilidad de la traslación en función de la altura de la ventana y el ángulo de visión para que la correspondencia entre píxel del *mouse* y unidad de posición del objeto sea válida.
- d. **_cb_mouse_pressed - _CB_MOUSE_PRESSED - click izquierdo:** Cuando se presiona el botón izquierdo y se mueve el *mouse*, para rotar el modelo 3D como una cámara de trackball.
- Verifica que el botón de la rueda del *mouse* esté oprimido
 - La rotación como resultado de la diferencia de la coordenada anterior del *mouse* y la actual, multiplicada por una constante de sensibilidad.
 - Llama a Reset que resetea la matriz de transformación
- e. **_cb_mouse_pressed - _CB_MOUSE_PRESSED - click rueda:** Cuando se oprime el botón de la rueda del *mouse* se resetea la posición del objeto.
- Verifica que el botón de la rueda del *mouse* esté oprimido
 - Llama a Reset que resetea la matriz de transformación
- f. **_cb_wheel:** Aunque intentamos implementar este callback, tuvimos problemas de compilación: no reconocía esta función dentro del .h de la librería. Esto no impidió que consiguiéramos agregar la funcionalidad a la ruedita del *mouse*. Lo hicimos dentro del callback de *mouse* motion, ahí podemos obtener que botón fue presionado por el usuario y se realizó lo siguiente:
- Si la rueda se mueve hacia adelante, se modifica el radio del eje z dando la impresión de que la cámara se mueve hacia delante.
 - Para el caso donde la rueda se mueve hacia atrás, se modifica el ángulo de tal forma que da la sensación de que la cámara se mueve hacia atrás.
 - Para ambos casos utilizamos un *step* que solo indica básicamente cuántas unidades se va a mover cada vez que se llama el callback.

Demostración en video

[Enlace](#)